PRO2000 - PROGRAMMING

Week 40- I/O, error handling

Gebremariam Assres
Gebremariam.assres@kristiania.no

Today's topics

- Input and Output Streams
- More Detailed Error Testing
- Member Functions for Reading and Writing Files
- Binary Files
- Creating Records with Structures
- Random-Access Files
- Opening a File for Both Input and Output
- **Databases and mysql-connector/c**

I/O Streams

- **Input Stream**
  - A stream from which data is **read**
  - Example: `cin` ← keyboard
  - Common classes: `istream`, `ifstream`, `istringstream`
- **Output Stream**
  - A stream to which data is **written**
  - Example: `cout` → monitor
  - Common classes: `ostream`, `ofstream`, `ostringstream`
- **Input/Output Stream**
  - Supports **both reading and writing**
  - Common class: `fstream`

I/O Streams …

- **File Stream Classes**
  - `Ifstream` –open primarily for input
  - `ofstream` -open primarily for output
  - `fstream` -open for either or both input and output
  - All three provide open() and close() member functions.
  - open() connects the program to a file.
  - close() disconnects the file from the program.
  - Keep files open only as long as necessary and always close them before the program ends.

I/O Streams ...

- File Open Modes:
  - **File open modes** specify how a file is opened and what operations are permitted.
  - Modes can be combined using |.
  - They are passed as the second argument to open().

`ios::app`	create new file, or append to end of existing file
`ios::ate`	go to end of existing file; write anywhere
`ios::binary`	read/write in binary mode (not text mode)
`ios::in`	open for input
`ios::out`	open for output

I/O Streams …

- Using fstream for Input or Output:
  - fstream can be configured for input or output depending on the open mode.
  - Input:

```
fstream file;
file.open("myfile.dat", ios::in);
```

  - Output:

```
fstream file;
file.open("myfile.dat", ios::out);
```

  - Using fstream for Both: fstream can support reading and writing simultaneously.

```
fstream file;
file.open("myfile.dat", ios::in | ios::out);
```

    - ios::in → enables reading
    - ios::out → enables writing
    - Not every combination of file modes is meaningful or appropriate.

I/O Streams ...

- Opening Files with Constructors:
  - File streams can be opened directly when the stream object is created.
  - fstream inFile("myfile.dat", ios::in);
  - Stream constructors accept the **same parameters as open()**.

  - This eliminates the need for a separate open() call.
  - ifstream and ofstream have **default open modes**, so specifying the mode is often optional.

I/O Streams ...

Example of a C++ program that:

- Opens an input file named input.txt.
- Reads the contents of the file.
- Creates/opens an output file named output.txt.
- Writes the contents read from input.txt into output.txt.
- Displays an error message if either file cannot be opened.

I/O Streams ...

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
   // Open input file using constructor
   ifstream inFile("input.txt", ios::in);

   // Create output stream object
   ofstream outFile;

   // Open output file using open()
   outFile.open("output.txt", ios::out);

   if (!inFile || !outFile)   {
      cout << "Error opening file." << endl;
      return 1;
   }
   char ch;

   // Read from input file and write to output file
   while (inFile.get(ch))
   {
      outFile << ch;
   }

   // Close files
   inFile.close();
   outFile.close();

   cout << "File copied successfully." << endl;

   return 0;
}
```

I/O Manipulators and output formatting

- Format output with I/O manipulators- using file objects like with **cout**
  - Can format with formatting member functions
  - **ostringstream**class allows in-memory formatting as a string object before writing to a file
  - I/O Manipulators:

Manipulators	**Description**
left, right	left or right justify output
oct, dec,hex	display output in octal, decimal, or hexadecimal
endl, flush	write newline (**endl** only) and flush output
showpos, noshowpos	do, do not show leading **+**with non-negative numbers
showpoint, noshowpoint	do, do not show decimal point and trailing zeroes
fixed, scientific	use fixed or scientific notation for floating-point numbers
setw(n)	sets minimum field output width to **n**
setprecision(n)	sets floating-point precision to **n**
setfill(ch)	uses **ch**as fill character

I/O Manipulators …

- **sstream** Formatting
  - To format output into an in-memory string object, include the **sstream** header file and create an **ostringstream**object

    ```
    #include <sstream> ostringstream outStr;
    ```
  - Write to the **ostringstream**object using I/O manipulators, all other stream member functions:

    ```
    outStr << showpoint << fixed
    << setprecision(2)
    << '$'<< amount;
    ```
  - Access the C-string inside the **ostringstream**object by calling its **str**member function

    ```
    cout << outStr.str();
    ```
- Error Testing:
  - Stream objects have error bits (flags) that are set by every operation to indicate success or failure of the operation, and the status of the stream
  - Stream member functions report on the settings of the flags

I/O Manipulators ...

- Error State Bits- can examine error state bits to determine file stream status

`ios::eofbit`	set when end of file detected
`ios::failbit`	set when operation failed
`ios::hardfail`	set when an irrecoverable error occurred
`ios::badbit`	set when invalid operation attempted
`ios::goodbit`	set when no other bits are set

- Error Bit Reporting Functions

`eof()`	true if **`eofbit`** set, false otherwise
`fail()`	true if **`failbit`** or **`hardfail`** set, false otherwise
`bad()`	true if **`badbit`** set, false otherwise
`good()`	true if **`goodbit`** set, false otherwise
`clear()`	clear all flags (no arguments), or clear a specific flag

I/O Manipulators ...

- Detecting File Operation Errors
  - The file handle is set to true if a file operation succeeds, false when it fails
  - Test the status of the stream by testing the file handle:

```
inFile.open("myfile");
if (!inFile)
{ cout << "Can't open file"; exit(1);}
```

- Member Functions for Reading and Writing Files
  - Unlike the extraction operator >>, these reading functions do not skip whitespace:

`getline`: read a line of input
`get`: reads a single character
`seekg`: goes to beginning of input file

- **getline** Member Function

```
getline(char s[ ], int max,charstop='\n')
```

`char s[ ]`: Character array to hold input
`int max`: 1 more than the maximum number of characters to read
`char stop`: Terminator to stop at if encountered before `max` number of characters is read. Optional, default is `'\n'`

I/O Manipulators …

- Single Character Input
  - Read a single character from the input stream and put it in **ch** (does not skip whitespace)- **get(char &ch)**.

```
ifstream inFile; char ch;
inFile.open("myFile"); inFile.get(ch);
cout << "Got " << ch;
```

  - Read a single character from the input stream and return (does not skip whitespace)- **get()**.

```
ifstream inFile; char ch;
inFile.open("myFile");
ch = inFile.get(); cout << "Got " << ch;
```

  - Read a single character from the input stream but do not remove the character from the input stream (does not skip whitespace)- **peek()**.

```
ifstream inFile; char ch; inFile.open("myFile");
ch = inFile.peek();
cout << "Got " << ch; ch = inFile.peek();
cout << "Got " << ch;//same output
```

I/O Manipulators …

- Single Character Output
  - Output a character to a file- `put(char ch)` - `outFile.put('G');`
  - Move **#bytes** to a given `offset` relative to `place` in file-`seekg(offset, place)`
    - `place`: location in file from which to compute offset- beginning(`ios::beg`), end(`ios::end`) and current position in file (`ios::cur`)
    - To move to the beginning of file, seek to an offset of zero from beginning of file inFile.seekg(0L, ios::beg);
    - Error or eof bits will block seeking to the beginning of file. Clear bits first: inFile.clear(); inFile.seekg(0L, ios::beg);
- Example of Single Character I/O
  - To copy an input file to an output file

```
char ch; infile.get(ch);
while (!infile.fail())
{ outfile.put(ch); infile.get(ch);}
infile.close(); outfile.close();
```

Binary Files

- Binary files store data in the computer's main memory format
  - Text files store the numeric data (0/1) converted into strings of ASCII characters
  - Files are opened in text mode (as text files) by default
- Using Binary Files
  - Pass **`ios::binary`** flag to the **`open`** member function to open a file in binary mode

    ```
    infile.open("myfile.dat",ios::binary);
    ```
  - Reading/writing of binary files requires special **`read`** and **`write`** member functions

    ```
    read(char *buffer, int numberBytes) write(char *buffer, int numberBytes)
    ```
- Using read and write
  - `buffer:` holds an array of bytes to transfer between memory and the file
  - `numberBytes`: the number of bytes to transfer
  - Address of buffer needs to be cast to `char*` using **`reinterpret_cast <char *>`**

```
read(char *buffer,int numberBytes) write(char *buffer,int numberBytes)
```

Binary Files ...

- Using write- to write an array of 2 doubles to a binary file

```
ofstream outFile("myfile",ios:binary);
double d[2] = {12.3, 34.5};
outFile.write(reinterpret_cast<char *>(d),sizeof(d));
```

- Using read- to read two 2 doubles from a binary file into an array

```
ifstream inFile("myfile", ios:binary); const int DSIZE = 10;
double data[DSIZE];
inFile.read( reinterpret_cast<char *>(data), 2*sizeof(double));
// only data[0] and data[1] contain values
```

Binary Files ...

- Creating Records with Structures
  - Can write structures to, read structures from files
  - To work with structures and files, use **`binary`** file flag upon open, and use **`read`**, **`write`** member functions

```
struct TestScore{
  int studentId;
  float score;
  char grade;
};
TestScore test1[20];
...
// write out test1 array to a file gradeFile.write(
  reinterpret_cast<char*>(test1), sizeof(test1));
```

Binary Files …

- File Access
  - Sequential access: Reads data in order from the beginning. To reach entry 100, read the previous 99 entries first.
  - Random access jumps directly to any file position has functions:
    - **seekg()** - moves the input/read position.
    - **seekp()** - moves the output/write position.
    - Syntax: **seekg(offset, place)** or **seekp(offset, place)- offset**: number of bytes to move; **place**: **ios::beg**, **ios::cur**, or **ios::end**

      ```
      inData.seekg(25L, ios::beg);   // Read 25 bytes after the beginning
      outData.seekp(-10L, ios::cur); // Write 10 bytes before the current position
      ```

Database

Databases

- Database vs files
  - Databases are better for large, shared, structured, and frequently changing data.
  - Files are suitable for small and simple data storage.

Databases	**Files**
Efficient searching and filtering	Can be slow with large data
Support multiple users simultaneously	Difficult to manage concurrent access
Provide data consistency and validation	Higher risk of duplicate or invalid data
Offer security and user permissions	Limited security controls
Support backup, recovery, and transactions	Usually require manual handling
Easily handle relationships between data	Relationships are harder to manage

Databases ...

- Using MySQL Database with C++
  - MySQL is a popular relational database management system.
  - C++ applications can connect to MySQL using **MySQL Connector/C**.
  - Common use cases: Desktop applications, Web backends, Inventory systems, Student and employee management systems, Data-driven games
  - Basic workflow. Install MySQL Server, Install MySQL Connector/C, Connect the C++ application to MySQL, Execute SQL queries, and Process results and close the connection.

Databases ...

- MySQL Connector/C
  - MySQL Connector/C provides the **MySQL C API**.
  - It can be used directly in C++ programs.
  - Main header file: #include <mysql/mysql.h>
- Typical workflow:
  - Initialize the MySQL library, Establish a connection, Execute SQL queries, Process results, and Close the connection.

Databases ...

- Connecting to MySQL

```
#include <iostream>
#include <mysql/mysql.h>

int main() {
    MYSQL* connection = mysql_init(nullptr);

    if (!connection) {
        std::cerr << "Initialization failed\n";
        return 1;
    }

    if (!mysql_real_connect(
            connection,
            "localhost",
            "app_user",
            "password",
            "company_db",
            3306,
            nullptr,
            0)) {
        std::cerr << mysql_error(connection) << '\n';
        mysql_close(connection);
        return 1;
    }

    std::cout << "Connected successfully!\n";

    mysql_close(connection);
    return 0;
}
```

Databases ...

- Executing a Query- Important Connector/C Functions
  - **mysql_init()** — initializes a connection object
  - **mysql_real_connect()** — connects to MySQL
  - **mysql_query()** — executes an SQL statement
  - **mysql_close()** — closes the connection

```
const char* query =
   "CREATE TABLE IF NOT EXISTS users ("
   "id INT AUTO_INCREMENT PRIMARY KEY,"
   "name VARCHAR(100),"
   "email VARCHAR(150)"
   ")";

if (mysql_query(connection, query) != 0) {
   std::cerr << mysql_error(connection) << '\n';
}
```

Databases ...

- Inserting data

```
const char* query = "INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com')";
if (mysql_query(connection, query) != 0) {
   std::cerr << "Insert failed: " << mysql_error(connection) << '\n';
} else { std::cout << "User inserted successfully.\n";}
```

- Retrieving data

```
mysql_query(connection, "SELECT id, name FROM users");
MYSQL_RES* results = mysql_store_result(connection);
MYSQL_ROW row;
while ((row = mysql_fetch_row(results))) {
   std::cout << row[0] << ": "
         << row[1] << '\n';
}

mysql_free_result(results);
```

Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on I/O, and error handling.*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)